

Spring Boot — Lecture 2

First Hello World App + IP, Port, DNS & Spring Initializr

1. Goal of This Lecture

Build a minimal Spring Boot app that returns **"Hello World"** in the browser. We won't understand the syntax yet — the intent is to experience how fast and easy Spring Boot makes setup, so you can jump straight to business logic.

What we hit	Where	Expected response
GET /hello	localhost:8080/hello	"Hello World" string in browser

2. IP Address

Every device on a network has a unique **IP address**. To check yours: Google 'what is my ip' → whatismyip.com → copy your IPv4 address.

Any client anywhere in the world that wants to reach your computer must send data to this specific IP address.

3. Port Number

One IP address can have many applications running simultaneously. The OS uses the **port number** to route incoming data to the correct app.

Application	Port
Spring Boot App	8080 (default)
HTTP traffic	80 (standard — auto-appended by browser)
HTTPS traffic	443 (standard — auto-appended by browser)
Chrome / WhatsApp / Spotify	other ports

Because standard ports (80/443) are auto-appended by the browser, you never type them. For local development you must explicitly type :8080.

4. Full Client → Server Request Flow

For a real website (e.g. coderarmyarmy.in):

Step	What Happens
1 — DNS Lookup	Browser asks DNS server: 'What is the IP of coderarmyarmy.in?' → gets server IP
2 — Port attach	Browser auto-appends port 443 (HTTPS) to the IP
3 — Packet travels	Data reaches server's IP:443
4 — Reverse Proxy	Maps port 443 → 8080 (where Spring Boot is running)
5 — Spring Boot	Receives request on port 8080, processes it, returns response

Reverse Proxy (e.g. Nginx):

Sits between client and your Spring Boot app. Maps the public-facing port (443) to the internal app port (8080). Both together are considered the 'server'.

For localhost (our case):

Term	Meaning
127.0.0.1	Fixed loopback IP — means 'this computer itself'
localhost	Alias for 127.0.0.1
localhost:8080/hello	Hit port 8080 on this machine, endpoint /hello

No DNS lookup needed when client and server are on the same machine.

5. Spring Initializr — start.spring.io

Without it, building a Java web app from scratch requires:

- Create proper folder structure
- Download all required dependencies (external libraries)
- Configure Spring Boot version
- Configure Java version
- Set up package structure

All of this before writing a single line of business logic! Spring Initializr generates a ready-made project skeleton (zip file) with all of it done.

Dependency = External Library your code depends on.

Example: To connect Java code to MySQL, you need the MySQL Connector library. Since your code depends on it → it becomes a **dependency**.

Choices made in this lecture:

Option	Choice	Why
Project	Maven	Most popular Java project management tool
Language	Java	—
Spring Boot version	2.1.6	SNAPSHOT (bugs possible). RC = Release Candidate (almost final). Pick stable.
Packaging	JAR	Modern Spring Boot uses JAR. WAR was old-style.
Config format	Properties	application.properties (vs YAML — both fine)
Java Version	21	LTS — stable and modern
Dependency	Spring Web	Enables REST APIs via Spring MVC + embeds Apache Tomcat

6. Generated Project Structure

After unzipping and opening in IntelliJ IDEA:

```
src/ main/ java/ com.example.demo/ DemoApplication.java ← Entry point (has  
main method) resources/ application.properties ← Config file test/ pom.xml ←  
Maven config + dependencies
```

DemoApplication.java has a main() method → execution starts here. Don't worry about @SpringBootApplication and SpringApplication.run() for now — we'll cover them later.

7. Writing the Hello World App

Step 1 — Create HelloController.java

Create a new Java class in com.example.demo package named HelloController:

```
@RestController public class HelloController { @GetMapping("/hello") public String hello() { return "Hello World"; } @GetMapping("/bye") public String greetBye() { return "Bye"; } }
```

What each annotation does (preview — full detail in later lectures):

Annotation	What it does
@RestController	Marks this class as a gateway for API endpoints. Spring Boot will look here for URL mappings.
@GetMapping("/hello")	Maps HTTP GET requests to /hello endpoint → calls this method automatically
@GetMapping("/bye")	Maps HTTP GET requests to /bye endpoint → calls greetBye() method

Key insight: The method name and endpoint name don't have to match. @GetMapping("/bye") calls greetBye() — the annotation does the mapping.

8. Running the App

Run DemoApplication → watch the console logs:

Log line	What it means
Tomcat initialized with port 8080	Embedded Tomcat servlet container starting up
Starting service [Tomcat]	Tomcat service engine coming online
Tomcat started on port(s): 8080	Server is now live and listening for requests

Now visit in browser:

```
localhost:8080/hello → Hello World localhost:8080/bye → Bye
```

Whitabel Error Page on unknown endpoints means: 'No mapping found for this URL' — the server IS running, just no handler registered for that path.

9. Changing the Port

In src/main/resources/application.properties:

```
server.port=9090
```

After restart: Tomcat starts on 9090. localhost:8080/hello → site can't be reached. localhost:9090/hello → Hello World.

10. Bonus — Returning HTML

```
return "Hello World";
```

Browser renders the HTML tag — Hello World appears as a heading. Spring Boot returns whatever the method returns as the response body.

11. Open Questions (Answered in Future Lectures)

Question	Will be covered in...
Who started the Tomcat server?	Spring Boot auto-configuration internals
How did /hello map to hello() method?	Spring MVC DispatcherServlet deep dive
What does @RestController really do?	Spring MVC annotations lecture
What does SpringApplication.run() do inside?	Spring Core + IoC container lecture
Why learn Spring MVC if Spring Boot does it all?	Because Boot uses MVC — can't debug without knowing internals

You CANNOT skip Spring Core / Spring MVC. Even @GetMapping and @RestController are MVC concepts. Spring Boot just auto-configures them — the skill is still MVC.

Quick Summary

What we did	How
Created project	Spring Initializr → downloaded zip → opened in IntelliJ
Added endpoint	Created HelloController.java with @RestController + @GetMapping
Ran the app	DemoApplication.main() → embedded Tomcat starts on 8080
Tested in browser	localhost:8080/hello → Hello World
Changed port	server.port=9090 in application.properties
Multiple endpoints	@GetMapping("/bye") → greetBye() returns 'Bye'

Next Lecture: Maven deep-dive + Project folder structure + Spring Boot auto-configuration internals